

Bash Parameter Expansion

Complete Reference — Bash 4.x

A complete reference for bash parameter expansion — the built-in string manipulation syntax that lets you transform variable values without calling external tools like sed, cut, or awk. All forms use the `${...}` syntax.

10 sections · Basic · Default Values · String Length · Substring Extraction · Prefix/Suffix Removal · Pattern Substitution · Case Modification · Indirection · Array Expansion · Quoting · Quick Reference Card

1. Basic Expansion

The simplest form — wraps a variable name in braces. Behaves identically to `$var` but is unambiguous when the variable name is adjacent to other characters.

```
${var}          # same as $var
${var}file      # unambiguous — $varfile would look up "varfile"
echo "${USER}name" # -> safwannname (if USER=safwan)
```

2. Default Values

Handle unset or empty variables without if/else blocks. The colon (:) before the operator means the form also triggers when the variable is *empty*. Without the colon, it only triggers when the variable is *unset*.

Form	Behaviour
<code>\${var:-default}</code>	Use default if var is unset OR empty
<code>\${var-default}</code>	Use default only if var is unset (empty is passed through)
<code>\${var:=default}</code>	Assign AND use default if var is unset or empty
<code>\${var=default}</code>	Assign AND use default only if var is unset
<code>\${var:+alt}</code>	Use alt if var IS set and non-empty (inverse of :-)
<code>\${var+alt}</code>	Use alt if var IS set, even if empty
<code>\${var:?message}</code>	Print message and exit if var is unset or empty
<code>\${var?message}</code>	Print message and exit only if var is unset

```
name=""
echo ${name:-"anonymous"}    # -> anonymous (empty triggers :-)
echo ${name-"anonymous"}     # ->          (empty, var is set, no trigger)

# Exit with a usage message if no argument is given:
file=${1:? "Usage: script.sh <filename>"}

# Safe default for optional flag:
debug=${DEBUG:-0}
```

3. String Length

Returns the number of characters in the variable's value, or the number of elements in an array.

```

${#var}          # number of characters in var
${#array[@]}     # number of elements in an array

str="hello"
echo ${#str}     # -> 5

fruits=("apple" "banana" "cherry")
echo ${#fruits[@]} # -> 3

```

4. Substring Extraction

Extracts a portion of the string by position. Indices are 0-based. Negative offsets count from the end — **a space before the minus is required** to avoid ambiguity with the `:` - default operator.

Form	Behaviour
<code>\${var:offset}</code>	From offset to end of string
<code>\${var:offset:length}</code>	From offset, for length characters
<code>\${var: -n}</code>	Last n characters (space before minus required)
<code>\${var: -n:l}</code>	l characters starting n from the end

```

str="hello world"
echo ${str:6}      # -> world
echo ${str:6:3}    # -> wor
echo ${str: -5}    # -> world
echo ${str: -5:3}  # -> wor

```

5. Prefix and Suffix Removal

Strips a pattern from the start (`#`, `##`) or end (`%`, `%%`) of the variable. Patterns use glob syntax (`*`, `?`, `[...]`), not regex. Single operator = shortest match; double operator = longest match.

Form	Behaviour
------	-----------

<code>\${var#pattern}</code>	Remove shortest prefix matching pattern
<code>\${var##pattern}</code>	Remove longest prefix matching pattern
<code>\${var%pattern}</code>	Remove shortest suffix matching pattern
<code>\${var%%pattern}</code>	Remove longest suffix matching pattern

```
path="/home/safwan/.config/starship.toml"

${path#/home/safwan/}    # -> .config/starship.toml
${path##*/}             # -> starship.toml    (after last /)
${path%.*}              # -> /home/safwan/.config/starship
${path%%.*}             # -> /home/safwan/    (from first dot)

# Get filename (equivalent to basename):
${path##*/}             # -> starship.toml

# Get directory (equivalent to dirname):
${path%/}               # -> /home/safwan/.config

# Strip $HOME from a target path (used in dotfiles install scripts):
target="$HOME/.config/starship.toml"
${target#$HOME/}        # -> .config/starship.toml
```

Note: The `#` and `##` operators are the most frequently used forms in real scripts — especially for path manipulation in install scripts and build pipelines. Prioritise these.

6. Pattern Substitution

Find-and-replace on variable values. Uses glob patterns, not regex. Leave the replacement empty to delete all occurrences.

Form	Behaviour
<code>\${var/pat/rep}</code>	Replace first match of pat with rep
<code>\${var//pat/rep}</code>	Replace all matches of pat with rep
<code>\${var/#pat/rep}</code>	Replace only if pat matches at the start
<code>\${var/%pat/rep}</code>	Replace only if pat matches at the end

```

str="hello world world"

echo ${str/world/bash}      # -> hello bash world    (first only)
echo ${str//world/bash}    # -> hello bash bash      (all)
echo ${str/#hello/hi}      # -> hi world world      (start match)
echo ${str/%world/bash}    # -> hello world bash    (end match)

# Delete all spaces:
echo ${str// /}            # -> helloworldworld

# Replace underscores with hyphens:
name="my_project_name"
echo ${name//_/-}          # -> my-project-name

```

7. Case Modification (bash 4+)

Requires bash 4 or later. macOS ships with bash 3.2, so these will silently fail there — one more reason to stay on Linux.

Form	Behaviour
<code>\${var^}</code>	Uppercase first character
<code>\${var^^}</code>	Uppercase all characters
<code>\${var,}</code>	Lowercase first character
<code>\${var,,}</code>	Lowercase all characters
<code>\${var~}</code>	Toggle case of first character
<code>\${var~~}</code>	Toggle case of all characters

```

str="hello WORLD"

echo ${str^}      # -> Hello WORLD
echo ${str^^}    # -> HELLO WORLD
echo ${str,,}    # -> hello world
echo ${str~}     # -> HELLO world

```

8. Indirection

Use the *value* of one variable as the name of another variable to look up. Also allows listing all variable names sharing a prefix.

```

${!var}          # use value of var as a variable name
${!prefix*}      # list all variable names starting with prefix
${!prefix@}      # same, but each name separately quoted

fruit="apple"
apple="red"
echo ${!fruit}    # -> red    (looked up $apple via $fruit)

# List all variables starting with BASH:
echo ${!BASH*}    # -> BASH BASHOPTS BASHPID BASH_ALIASES ...

```

9. Array Expansion

Parameter expansion extends naturally to indexed and associative arrays. Always use `[@]` rather than `[*]` in loops — the former keeps each element as a separate word; the latter joins them into one.

Form	Behaviour
<code>\${array[@]}</code>	All elements, each as a separate word — use in loops
<code>\${array[*]}</code>	All elements as a single concatenated word
<code>\${#array[@]}</code>	Number of elements
<code>\${!array[@]}</code>	All indices (indexed) or keys (associative)
<code>\${array[@]:off:len}</code>	Array slice: len elements starting at off

```

fruits=("apple" "banana" "cherry")

echo ${fruits[@]}          # -> apple banana cherry
echo ${#fruits[@]}        # -> 3
echo ${!fruits[@]}        # -> 0 1 2
echo ${fruits[@]:1:2}      # -> banana cherry

# Iterating correctly:
for f in "${fruits[@]}"; do echo "$f"; done

# Associative array keys:
declare -A cfg=( [host]="localhost" [port]="8080" )
echo ${!cfg[@]}           # -> host port

```

10. Quoting and Introspection (bash 4.4+)

Transform variables into forms safe for reuse as shell input, or inspect their attributes. Less common in day-to-day scripting but useful when building tools that generate shell code.

Form	Behaviour
<code>\${var@Q}</code>	Quoted form safe to reuse as shell input
<code>\${var@E}</code>	Expand escape sequences (<code>\n</code> , <code>\t</code> , etc.)
<code>\${var@A}</code>	Output as an assignment statement (declare -p style)
<code>\${var@a}</code>	Attribute flags of the variable (<code>r=readonly</code> , <code>x=exported</code>)

```

str="hello world"
echo ${str@Q}      # -> 'hello world' (properly shell-quoted)

path=$'line1\nline2'
echo "${path@E}"   # -> line1
                  #   line2

```

11. Quick Reference Card

Forms marked ★ **memorise** are the ones you will reach for regularly in real scripts. The rest are reference material — know they exist and look them up when needed.

Form	What it does
------	--------------

<code>\${var}</code> ★	Basic expansion — unambiguous adjacent to text
<code>\${#var}</code> ★	String length
<code>\${var:-val}</code> ★	Default if unset or empty
<code>\${var:=val}</code>	Assign default if unset or empty
<code>\${var:+val}</code>	Alternate value if set and non-empty
<code>\${var:?msg}</code>	Exit with message if unset or empty
<code>\${var:n}</code>	Substring from index n
<code>\${var:n:l}</code>	Substring from n, length l
<code>\${var#pat}</code> ★	Strip shortest prefix match
<code>\${var##pat}</code> ★	Strip longest prefix match
<code>\${var%pat}</code> ★	Strip shortest suffix match
<code>\${var%%pat}</code>	Strip longest suffix match
<code>\${var/pat/rep}</code>	Replace first match
<code>\${var//pat/rep}</code> ★	Replace all matches
<code>\${var/#pat/rep}</code>	Replace only if at start
<code>\${var/%pat/rep}</code>	Replace only if at end
<code>\${var^}</code> ★	Uppercase first character
<code>\${var^^}</code> ★	Uppercase all characters
<code>\${var,,}</code> ★	Lowercase all characters
<code>\${var,}</code>	Lowercase first character
<code>\${var~~}</code>	Toggle case of all characters
<code>\${!var}</code>	Variable indirection
<code>\${!prefix*}</code>	Variable names matching prefix
<code>\${var@Q}</code>	Quoted form safe for shell reuse

12. The One Rule to Remember

Always quote your expansions.

```
"${var}"    # correct – immune to word splitting and glob expansion  
${var}     # dangerous – spaces or glob chars in value cause bugs
```

Unquoted expansions are subject to word splitting and pathname expansion. A path like `/home/user/my project/` becomes two arguments without quotes. Always write `"${var}"`, not `${var}`, inside scripts.